



Study Material

MODULE 3: INHERITANCE and PACKAGES (9 HOURS)

Contents

1. Inheritance in Java	2
2. Types of Inheritance in Java	3
2.1. Single Inheritance	3
2.2. Multilevel Inheritance	5
2.3. Hierarchical Inheritance	7
3. Superclass and Subclass	11
4. super Keyword in Java	13
5. final Keyword in Java.....	15
6. Dynamic Method Dispatch in Java.....	17
7. Abstract Classes and Abstract Methods.....	18
8. Interfaces in Java	20
9. Packages in Java	24
10. Creating a Package in Java	25
11. Importing Packages in Java	27
PRACTICE QUESTION.....	30



1. Inheritance in Java

Definition

Inheritance in Java is an object-oriented programming mechanism by which one class acquires the properties (data members) and behaviors (methods) of another class. It supports code reusability, improves program structure, and establishes a parent–child relationship between classes.

- Parent class is known as the Superclass
- Child class is known as the Subclass
- The keyword used to achieve inheritance in Java is extends

Key Features of Inheritance

- Enables code reuse by allowing subclasses to use existing superclass methods
- Supports method overriding, enabling runtime polymorphism
- Improves maintainability and scalability of applications
- Represents IS-A relationship (e.g., Dog *is a* Animal)

Syntax of Inheritance

```
class SuperClass {  
    // variables and methods  
}
```

```
class SubClass extends SuperClass {  
    // additional variables and methods  
}
```

Explanation of Syntax

- SuperClass contains common properties and methods
- SubClass inherits all non-private members of SuperClass
- The subclass can:
 - Use inherited methods directly
 - Add new methods and variables
 - Override superclass methods

Example Program

```
class Animal {  
    void eat() {  
        System.out.println("Animal eats food");  
    }  
}
```



```
}  
}
```

```
class Dog extends Animal {  
    void bark() {  
        System.out.println("Dog barks");  
    }  
}
```

```
public class TestInheritance {  
    public static void main(String[] args) {  
        Dog d = new Dog();  
        d.eat(); // inherited method  
        d.bark(); // subclass method  
    }  
}
```

Output:

Animal eats food

Dog barks

Important Points to Remember

- Java supports single, multilevel, and hierarchical inheritance using classes
- Multiple inheritance is not supported using classes to avoid ambiguity
- Constructors are not inherited, but superclass constructors can be called using super
- Private members of a superclass cannot be accessed directly in a subclass

2. Types of Inheritance in Java

2.1. Single Inheritance

Definition

Single inheritance is a type of inheritance in which one subclass inherits from exactly one superclass. This is the simplest and most commonly used form of inheritance in Java.

It establishes a one-to-one parent–child relationship between classes.



Key Characteristics

- Only one parent class and one child class
- Achieved using the extends keyword
- Promotes code reuse
- Easy to understand and maintain
- Supports method overriding and polymorphism

Syntax

```
class SuperClass {  
    // data members and methods  
}
```

```
class SubClass extends SuperClass {  
    // additional methods and variables  
}
```

Example Program

```
class A {  
    void show() {  
        System.out.println("Class A");  
    }  
}
```

```
class B extends A {  
    void display() {  
        System.out.println("Class B");  
    }  
}
```

```
public class SingleInheritanceDemo {  
    public static void main(String[] args) {  
        B obj = new B();  
        obj.show();    // inherited method  
    }  
}
```



```
        obj.display(); // subclass method
    }
}
```

Output

Class A

Class B

Explanation

- Class A is the superclass
- Class B is the subclass
- Method show() is inherited by class B
- Object of class B can access both superclass and subclass methods

Advantages of Single Inheritance

- Improves code reusability
- Reduces redundancy
- Makes programs easier to debug and maintain
- Supports runtime polymorphism

2.2. Multilevel Inheritance

Definition

Multilevel inheritance is a type of inheritance in which a class is derived from another derived class, forming a chain of inheritance. In this hierarchy, each subclass inherits the features of its immediate parent as well as all higher-level superclasses.

Key Characteristics

- Inheritance occurs in multiple levels
- Forms a parent → child → grandchild relationship
- Achieved using the extends keyword
- Supports method overriding and runtime polymorphism
- Encourages reusability across multiple levels

Structure

Class A → Class B → Class C

- Class A: Superclass
- Class B: Subclass of A, Superclass of C
- Class C: Subclass of B



Syntax Example

```
class A {  
    void showA() {  
        System.out.println("A");  
    }  
}
```

```
class B extends A {  
    void showB() {  
        System.out.println("B");  
    }  
}
```

```
class C extends B {  
    void showC() {  
        System.out.println("C");  
    }  
}
```

Main Program

```
public class MultilevelInheritanceDemo {  
    public static void main(String[] args) {  
        C obj = new C();  
        obj.showA(); // inherited from class A  
        obj.showB(); // inherited from class B  
        obj.showC(); // class C method  
    }  
}
```

Output

A

B

C



Explanation

- Class C indirectly inherits methods from class A through class B
- An object of the lowest-level subclass can access all public and protected members of its superclasses
- Demonstrates hierarchical reuse of functionality

Advantages of Multilevel Inheritance

- Promotes extensive code reuse
- Improves logical program structure
- Helps represent real-world hierarchical relationships
- Supports dynamic method dispatch

Limitations

- Increases program complexity
- Debugging becomes harder in deep inheritance chains
- Tight coupling between classes

2.3. Hierarchical Inheritance

Definition

Hierarchical inheritance is a type of inheritance in which multiple subclasses inherit from a single superclass. In this model, one parent class serves as the base for two or more child classes.

Key Characteristics

- One superclass, multiple subclasses
- Promotes shared functionality across different child classes
- Implemented using the extends keyword
- Supports method overriding and runtime polymorphism
- Improves code organization and reuse

Structure

Class A

/ \

Class B Class C

Syntax Example

```
class A {  
    void show() {  
        System.out.println("A");  
    }  
}
```



```
}  
}
```

```
class B extends A {  
    void displayB() {  
        System.out.println("B");  
    }  
}
```

```
class C extends A {  
    void displayC() {  
        System.out.println("C");  
    }  
}
```

Main Program

```
public class HierarchicalInheritanceDemo {  
    public static void main(String[] args) {  
        B objB = new B();  
        C objC = new C();  
  
        objB.show();    // inherited from class A  
        objB.displayB(); // class B method  
  
        objC.show();    // inherited from class A  
        objC.displayC(); // class C method  
    }  
}
```

Output

```
A  
B  
A
```




C

Explanation

- Class A acts as a common base class
- Classes B and C independently inherit from A
- Changes in superclass A are automatically reflected in all subclasses
- Subclasses cannot access each other's methods directly

Advantages of Hierarchical Inheritance

- Reduces code duplication
- Enhances maintainability
- Useful when multiple classes share common behavior
- Reflects real-world relationships (e.g., Vehicle → Car, Bike)

Limitations

- Increased dependency on the superclass
- Improper design can lead to tight coupling
- Complex hierarchy may reduce readability

Here is an extended, exam-oriented explanation of Multiple Inheritance in Java, written clearly for theory answers (5–10 marks).

Multiple Inheritance in Java

Definition

Multiple inheritance is a type of inheritance in which a single subclass inherits from more than one superclass at the same time.

Why Java Does NOT Support Multiple Inheritance Using Classes

Java does not allow multiple inheritance through classes to avoid ambiguity and complexity, mainly due to the Diamond Problem.

Diamond Problem (Ambiguity Issue)

Consider the following scenario:

```
class A {  
    void show() {  
        System.out.println("Class A");  
    }  
}
```



```
class B extends A {  
    void show() {  
        System.out.println("Class B");  
    }  
}
```

```
class C extends A {  
    void show() {  
        System.out.println("Class C");  
    }  
}
```

// This is NOT allowed in Java

```
class D extends B, C {  
}
```

Question:

If object D calls show(), which version should be executed—B's or C's? This creates confusion for the compiler, known as the diamond ambiguity problem.

Hence, Java Restriction

- Java does not allow a class to extend more than one class
- This ensures:
 - Clear method resolution
 - Simpler and safer object model
 - Better maintainability

Multiple Inheritance Using Interfaces (Supported)

Java supports multiple inheritance using interfaces because:

- Interfaces contain abstract methods only
- The implementing class must provide method definitions
- No ambiguity arises

Example Using Interfaces

```
interface A {
```



```
void show();  
}
```

```
interface B {  
    void display();  
}
```

```
class C implements A, B {  
    public void show() {  
        System.out.println("Show method");  
    }  
  
    public void display() {  
        System.out.println("Display method");  
    }  
}
```

Key Points to Remember

Aspect	Classes	Interfaces
Multiple inheritance	Not allowed	Allowed
Method ambiguity	Yes	No
Keyword used	extends	implements
Method implementation	Allowed	Provided by class

3. Superclass and Subclass

Superclass

A superclass is the parent class in an inheritance relationship. It contains common data members and methods that can be shared by its subclasses.

Characteristics of Superclass:

- Acts as a base or parent class
- Provides common properties and behaviors
- Its public and protected members can be accessed by subclasses



- Helps in achieving code reusability

Subclass

A subclass is the child class that inherits from a superclass using the extends keyword. It can reuse, modify, or extend the functionality of the superclass.

Characteristics of Subclass:

- Acts as a derived or child class
- Uses the extends keyword to inherit another class
- Can add new methods and variables
- Can override existing methods of the superclass
- Represents an IS-A relationship

Syntax Example

```
class SuperClass {  
    void show() {  
        System.out.println("Superclass method");  
    }  
}
```

```
class SubClass extends SuperClass {  
    void display() {  
        System.out.println("Subclass method");  
    }  
}
```

Explanation

- SuperClass defines common functionality
- SubClass automatically inherits the show() method
- SubClass adds its own method display()

Important Points

- A subclass can access public and protected members of its superclass
- Private members of a superclass cannot be accessed directly
- Constructors are not inherited, but can be invoked using super
- One superclass can have multiple subclasses



4. super Keyword in Java

Definition

The super keyword in Java is a reference variable that refers to the immediate superclass object. It is mainly used to differentiate between superclass and subclass members when they have the same name.

Uses of super Keyword

The super keyword is used to:

1. Access superclass variables
2. Call superclass methods
3. Call superclass constructors

1. Accessing Superclass Variables

When a subclass and superclass have variables with the same name, super is used to access the superclass variable.

Example

```
class A {  
    int x = 10;  
}  
  
class B extends A {  
    int x = 20;  
  
    void show() {  
        System.out.println(super.x); // Accesses superclass variable  
        System.out.println(x);      // Accesses subclass variable  
    }  
}
```

Output

10

20

2. Calling Superclass Methods

If a subclass overrides a method of the superclass, super can be used to call the overridden method of the superclass.

Example



```
class A {  
    void display() {  
        System.out.println("Display from A");  
    }  
}
```

```
class B extends A {  
    void display() {  
        super.display(); // calls superclass method  
        System.out.println("Display from B");  
    }  
}
```

3. Calling Superclass Constructor

`super()` is used to invoke the constructor of the superclass. It must be the first statement in the subclass constructor.

Example

```
class A {  
    A() {  
        System.out.println("Superclass Constructor");  
    }  
}
```

```
class B extends A {  
    B() {  
        super(); // calls A's constructor  
        System.out.println("Subclass Constructor");  
    }  
}
```

Important Rules

- `super` refers only to the immediate parent class
- `super()` must be the first statement in a constructor



- If super() is not written explicitly, Java inserts it automatically
- super cannot be used in a static context

Advantages of super Keyword

- Resolves naming conflicts
- Allows access to overridden methods
- Ensures proper constructor chaining
- Improves clarity and maintainability

5. final Keyword in Java

Definition

The final keyword in Java is used to restrict modification. It can be applied to variables, methods, and classes to prevent changes such as reassignment, overriding, or inheritance.

Uses of final Keyword

Usage	Meaning
-------	---------

final variable	Value cannot be changed once assigned
----------------	---------------------------------------

final method	Method cannot be overridden
--------------	-----------------------------

final class	Class cannot be inherited
-------------	---------------------------

1. final Variable

A variable declared as final becomes a constant. Its value cannot be modified after initialization.

Example

```
class Test {  
    final int MAX = 100;  
  
    void show() {  
        // MAX = 200; Error  
        System.out.println(MAX);  
    }  
}
```

2. final Method

A method declared as final cannot be overridden by subclasses. This is used to prevent modification of important methods.

Example



```
class A {  
    final void show() {  
        System.out.println("Final method");  
    }  
}
```

```
class B extends A {  
    // void show() { } Error  
}
```

3. final Class

A class declared as final cannot be inherited. This prevents further subclassing.

Example

```
final class A {  
    void show() {  
        System.out.println("Final class");  
    }  
}
```

```
// class B extends A Error
```

Important Points to Remember

- A final variable must be initialized
- A final method can be inherited but not overridden
- A final class cannot be extended
- final keyword improves security and performance
- Common example: String class is final in Java

Advantages of final Keyword

- Prevents accidental modification
- Ensures consistent behavior
- Improves program reliability
- Helps compiler optimization



6. Dynamic Method Dispatch in Java

Definition

Dynamic Method Dispatch is a mechanism by which the method call is resolved at runtime rather than at compile time. The method to be executed is determined based on the actual object type, not the reference type.

It is a key concept used to achieve runtime polymorphism in Java.

Key Characteristics

- Occurs at runtime
- Achieved using method overriding
- Depends on object reference
- Supports polymorphic behavior
- Involves inheritance

Conditions for Dynamic Method Dispatch

1. There must be an inheritance relationship
2. The method must be overridden in the subclass
3. The superclass reference must refer to a subclass object

Example Program

```
class A {  
    void show() {  
        System.out.println("Class A");  
    }  
}  
  
class B extends A {  
    void show() {  
        System.out.println("Class B");  
    }  
}  
  
public class Test {  
    public static void main(String[] args) {  
        A obj = new B(); // reference of A, object of B
```



```
        obj.show();    // calls B's show()
    }
}
```

Output

Class B

Explanation

- obj is a reference of class A
- At runtime, obj refers to an object of class B
- Java decides which show() method to execute during execution
- Hence, B's show() method is called

Why It Is Called “Dynamic”?

- The decision is made dynamically at runtime
- Enables flexible and extensible programs

Advantages

- Supports runtime polymorphism
- Allows method flexibility
- Improves code extensibility
- Enables loose coupling

Limitations

- Does not apply to static methods
- Does not work with final methods
- Constructors are not overridden

7. Abstract Classes and Abstract Methods

Abstract Class

Definition

An abstract class in Java is a class that is declared using the abstract keyword and cannot be instantiated (objects cannot be created directly). It is used to define a common base structure for its subclasses.

Characteristics of an Abstract Class

- Declared using the abstract keyword
- Cannot be instantiated
- May contain abstract methods (without body)



- May also contain non-abstract (concrete) methods
- Can have constructors, variables, and static methods
- Supports inheritance and polymorphism

Abstract Method

Definition

An abstract method is a method that does not have a body. It only declares the method signature and forces subclasses to provide the implementation.

Characteristics of Abstract Method

- Declared using the abstract keyword
- Contains no method body
- Must be overridden in the subclass
- Cannot be final, static, or private

Syntax Example

```
abstract class Shape {  
    abstract void draw(); // abstract method  
  
    void display() {      // non-abstract method  
        System.out.println("Shape class");  
    }  
}
```

```
class Circle extends Shape {  
    void draw() {  
        System.out.println("Drawing Circle");  
    }  
}
```

Main Program

```
public class AbstractDemo {  
    public static void main(String[] args) {  
        Shape s = new Circle(); // dynamic method dispatch  
        s.draw();  
    }  
}
```



```
s.display();  
}  
}
```

Output

Drawing Circle

Shape class

Explanation

- Shape is an abstract class and cannot be instantiated
- draw() is abstract and implemented in class Circle
- Abstract class reference can refer to subclass object
- Supports runtime polymorphism

Advantages of Abstract Classes

- Enforces method implementation in subclasses
- Provides partial abstraction
- Supports code reuse
- Improves program design and flexibility

Important Rules

- If a class contains an abstract method, it must be declared abstract
- A subclass must override all abstract methods or itself be abstract
- Abstract classes can have constructors
- Multiple inheritance is not supported using abstract classes

8. Interfaces in Java

Definition

An interface in Java is a collection of abstract methods that defines a contract for classes. Any class that implements an interface must provide implementations for all its abstract methods.

Interfaces are used to achieve full abstraction and multiple inheritance in Java.

Key Features of Interfaces

- Supports multiple inheritance
- Methods are public and abstract by default
- Variables are public static final (constants)
- Cannot be instantiated



- Implemented using the implements keyword

Syntax

```
interface A {  
    void show();  
}
```

```
class B implements A {  
    public void show() {  
        System.out.println("Interface implemented");  
    }  
}
```

Main Program

```
public class InterfaceDemo {  
    public static void main(String[] args) {  
        A obj = new B();  
        obj.show();  
    }  
}
```

Output

Interface implemented

Explanation

- Interface A contains an abstract method show()
- Class B implements interface A
- The method show() must be declared public
- Interface reference can refer to implementing class object
- Supports runtime polymorphism

Multiple Inheritance Using Interfaces

Java allows a class to implement multiple interfaces, avoiding ambiguity.

```
interface A {  
    void showA();  
}
```



```
interface B {  
    void showB();  
}
```

```
class C implements A, B {  
    public void showA() {  
        System.out.println("From A");  
    }  
  
    public void showB() {  
        System.out.println("From B");  
    }  
}
```

Interface vs Abstract Class

Feature	Interface	Abstract Class
Multiple inheritance	Yes	No
Method body	(Java 7)	Allowed
Variables	Constants only	Any type
Keyword used	implements	extends

Advantages of Interfaces

- Supports multiple inheritance
- Provides 100% abstraction
- Ensures loose coupling
- Improves program design and flexibility

Important Rules

- A class must implement all methods of an interface
- Interfaces cannot have constructors
- An interface can extend another interface
- A class can extend one class and implement multiple interfaces



Multiple Inheritance Using Interface in Java

Definition

Multiple inheritance is a mechanism where a single class inherits properties and behaviors from more than one parent. Although Java does not support multiple inheritance using classes, it supports multiple inheritance using interfaces.

Why Interfaces Allow Multiple Inheritance

- Interfaces contain only abstract methods (no implementation conflicts)
- The implementing class provides method definitions
- This avoids the diamond problem ambiguity

Syntax Example

```
interface A {  
    void showA();  
}
```

```
interface B {  
    void showB();  
}
```

```
class C implements A, B {  
    public void showA() {  
        System.out.println("Method from Interface A");  
    }  
  
    public void showB() {  
        System.out.println("Method from Interface B");  
    }  
}
```

Main Program

```
public class MultipleInheritanceDemo {  
    public static void main(String[] args) {  
        C obj = new C();  
    }  
}
```



```
        obj.showA();
        obj.showB();
    }
}
```

Output

Method from Interface A

Method from Interface B

Explanation

- Interface A and B declare abstract methods
- Class C implements both interfaces
- Class C provides implementations for all interface methods
- No ambiguity occurs because methods are defined in the class

Advantages

- Achieves multiple inheritance safely
- Supports loose coupling
- Improves flexibility and scalability
- Promotes interface-based programming

Important Rules

- A class can implement multiple interfaces
- All interface methods must be implemented
- Method implementations must be public
- A class can extend one class and implement multiple interfaces

9. Packages in Java

Definition

A package in Java is a collection of related classes and interfaces grouped together under a single name. Packages are used to organize code, improve maintainability, and provide access control.

Advantages of Packages

1. Code Organization - Packages help in arranging large Java programs into logical units, making the code easier to understand and manage.
2. Reusability - Classes inside a package can be reused in other programs by importing the package.



3. Avoids Naming Conflicts - Packages prevent class name duplication by grouping classes under unique package names.
4. Access Protection - Packages support access control using access modifiers such as public, protected, and default access.

Syntax

```
package mypackage;
```

Example

```
package mypack;
```

```
public class Test {  
    public void show() {  
        System.out.println("Package Example");  
    }  
}
```

Real-World Example

- java.util → Utility classes (Scanner, Date)
- java.io → Input/Output classes
- java.lang → Core classes (String, Math)

Important Points

- A package must be declared at the top of the source file
- Java provides both built-in and user-defined packages
- Packages improve security and modularity

Here is an extended, exam-ready explanation of Creating a Package in Java, written in clear academic style and suitable for 5–10 mark answers.

10. Creating a Package in Java

Definition

Creating a package in Java means grouping related classes and interfaces under a common namespace using the package keyword. User-defined packages help in organizing large applications and improving code reusability.

Steps to Create a Package

Step 1: Declare the Package

The package statement must be written as the first line in the Java source file.



```
package mypack;
```

```
public class A {  
    public void show() {  
        System.out.println("Package Example");  
    }  
}
```

Step 2: Compile the Program

Use the javac compiler with the -d option to create the directory structure for the package.

```
javac -d . A.java
```

Explanation of Compilation Command

- javac → Java compiler
- -d → Specifies destination directory
- . → Current directory
- A.java → Source file name

This command automatically creates a folder named mypack and stores the compiled .class file inside it.

Directory Structure Created

```
mypack  
└── A.class
```

Using the Package Class

```
import mypack.A;
```

```
class Test {  
    public static void main(String[] args) {  
        A obj = new A();  
        obj.show();  
    }  
}
```

Output

Package Example



Advantages of Creating Packages

- Organizes code logically
- Avoids class name conflicts
- Provides access protection
- Makes large projects manageable

Important Points

- Package name is usually written in lowercase
- One source file can belong to only one package
- Package declaration must come before import statements

11. Importing Packages in Java

Definition

Importing a package in Java allows classes and interfaces defined in one package to be used in another program without writing the fully qualified name every time.

The import keyword is used to access package members.

Types of Import Statements

1. Import a Single Class

Used when only one specific class is required from a package.

```
import mypack.A;
```

Example

```
import mypack.A;
```

```
class Test {  
    public static void main(String[] args) {  
        A obj = new A();  
        obj.show();  
    }  
}
```

2. Import Entire Package

Used when multiple classes are required from a package.

```
import mypack.*;
```

Example



```
import mypack.*;
```

```
class Test {  
    public static void main(String[] args) {  
        A obj = new A();  
        obj.show();  
    }  
}
```

Without Import Statement

Classes can also be accessed using the fully qualified name.

```
class Test {  
    public static void main(String[] args) {  
        mypack.A obj = new mypack.A();  
        obj.show();  
    }  
}
```

Important Points

- import statement must be written after the package statement
- Importing a package does not include sub-packages
- java.lang package is imported automatically
- Importing improves code readability

Advantages of Importing Packages

- Reduces code length
- Improves clarity
- Enables reuse of classes
- Makes programs easier to maintain

12. Member Access in Packages (Access Modifiers)

Definition

Access modifiers in Java are keywords used to control the visibility and accessibility of classes, methods, variables, and constructors. They play a crucial role in package-level access control and data security.



Types of Access Modifiers in Java

Java provides four access levels:

Modifier	Same Class	Same Package	Subclass	Outside Package
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default (no modifier)	✓	✓	✗	✗
private	✓	✗	✗	✗

1. public Access Modifier

- Members are accessible everywhere
- Can be accessed within the class, package, subclasses, and outside packages

```
public int x;
```

2. protected Access Modifier

- Accessible within the same package
- Accessible in subclasses outside the package
- Not accessible to non-subclasses outside the package

```
protected int y;
```

3. Default (Package-Private) Access

- No keyword used
- Accessible only within the same package
- Not accessible outside the package

```
int z; // default access
```

4. private Access Modifier

- Most restrictive
- Accessible only within the same class
- Used for data hiding

```
private int a;
```

Example Demonstrating Access Control

```
package p1;
```

```
public class A {
```



```
public int x = 10;
protected int y = 20;
int z = 30;
private int w = 40;
}
```

- x → accessible everywhere
- y → accessible in same package and subclasses
- z → accessible only in same package
- w → accessible only in class A

Why Access Modifiers Are Important

- Provide data security
- Support encapsulation
- Control class interaction
- Improve maintainability

PRACTICE QUESTION

1. Define inheritance.
2. What is a package?
3. What is dynamic method dispatch?
4. Define interface.
5. What is final keyword?
6. Explain types of inheritance in Java.
7. Explain super keyword with example.
8. Difference between abstract class and interface.
9. Explain packages and their advantages.
10. Explain inheritance in Java with types and examples.
11. Explain dynamic method dispatch with suitable program.
12. Explain abstract classes and interfaces with examples.
13. Explain creation, importing, and access control of packages.